Time-Series Databases

downsampling, retention & cardinality

Watch the 3-min video

In one line: a time-series database is built for append-heavy, timestamped data: it splits data into time chunks, compresses columns hard, rolls old points up into summaries, and deletes old data by dropping whole chunks. The one thing that breaks it is high-cardinality labels.

1 The problem: billions of tiny rows

You monitor **10,000 servers** and record CPU usage every minute. That is 10,000 × 1,440 minutes = **14.4 million rows a day**, for one metric, and over **5 billion a year**. Dashboards want the last hour, instantly. Plain Postgres starts to groan: the table and its indexes grow without end, and deleting old rows is slow.

Time-series data has a very clear shape:

- **Append-heavy.** New points arrive all the time; old ones are almost never updated.
- **Every point has a timestamp**, plus a few labels (host, region) and a value.
- **Queries ask for a time range** and usually aggregate: averages, maximums, percentiles.
- **Recent data is hot**; old data is rarely read.

Examples: server metrics, sensor readings, stock prices, application events. A time-series database (TSDB) is designed around exactly this shape.

2 Storage: time chunks and column compression

A TSDB **partitions data into time chunks**. TimescaleDB, a Postgres extension, calls the table a **hypertable** and splits it by time automatically, for example one chunk per day or week. A query for the last hour only touches the newest chunk, and the newest chunk's indexes fit in memory.

Inside older chunks, data is stored in **compressed columns**. Timestamps arrive at regular intervals, so they are stored as small deltas (often the same delta over and over). Values change slowly, so they are stored with bit tricks such as XOR against the previous value. Prometheus uses only about **1–2 bytes per sample**, and column compression commonly saves over 90%.

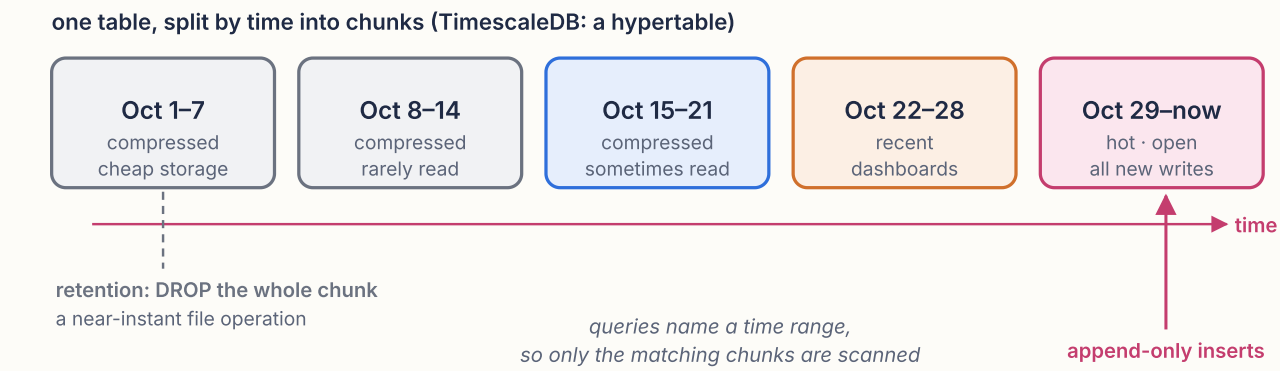


Figure 1. Data is chunked by time. New writes go to the open chunk; old chunks are compressed, moved to cheaper storage, or dropped whole.

3 Downsampling and retention

Downsampling keeps storage under control. Keep raw one-minute points for a week. Then roll them up into an **hourly minimum, average and maximum**, kept for a year. Nobody needs per-minute detail from last March; they need the trend and the peaks. In TimescaleDB, a **continuous aggregate** keeps that rollup up to date for you as new data arrives. Prometheus uses **recording rules** for the same idea: precompute an expression on a schedule and store the result as a new series.

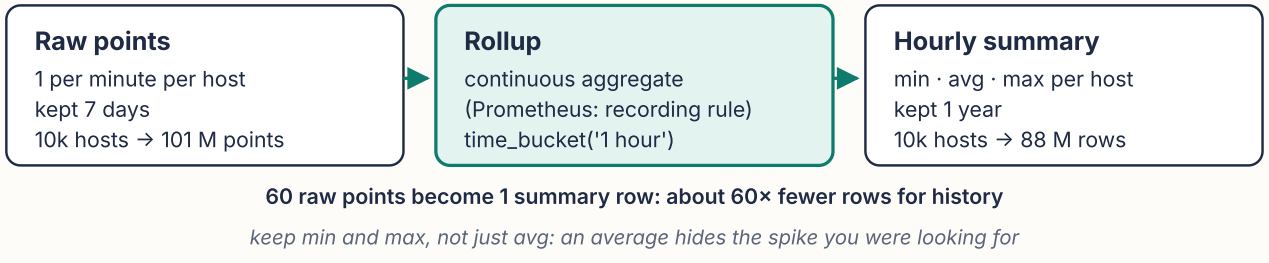


Figure 2. Raw points for a short window, summaries for the long one. History costs about 60× fewer rows.

Retention is just as simple. Because data is chunked by time, deleting last month means **dropping whole chunks**: a near-instant file operation. In plain Postgres, deleting billions of rows means a slow `DELETE`, table bloat, and heavy vacuuming afterwards. Older chunks can also be compressed harder, or moved to cheaper storage.

WORKED EXAMPLE: STORAGE FOR ONE METRIC

- **Raw for a week:** 14.4 M points/day × 7 = about **101 M points**.
- **Hourly for a year:** 10,000 hosts × 24 × 365 = about **88 M summary rows** (each holding min, avg, max).
- **Compressed:** a full year of raw points (5.3 B) at about 1.5 bytes each is roughly **8 GB**. The same data as ordinary Postgres rows, with per-row overhead and an index, is in the hundreds of GB.
- **Retention:** every day, one chunk older than 7 days is dropped. No vacuum storm.

4 The trap: high cardinality

Every unique combination of label values, like `host` and `region`, is a separate **series**, with its own entry in the index and its own buffer in memory. Add a label like `user_id` or `request_id`, and millions of series appear overnight. Memory and index size explode.

The rule: **keep labels bounded** (host, region, status code, endpoint), and put unique IDs in **logs or traces** instead. In Prometheus, the number of **active series** is the main driver of memory use.



Figure 3. Labels multiply. A bounded label grows the series count by a fixed factor; an unbounded one makes it unbounded.

5 In practice: a query and the popular choices

A `cpu` table as a hypertable. `time_bucket` groups points into five-minute buckets, per host, for the last hour:

```
SELECT create_hypertable('cpu', 'time');   -- chunk the table by time

SELECT time_bucket('5 minutes', time) AS bucket, host,
       avg(usage) AS avg_cpu, max(usage) AS max_cpu
  FROM cpu
 WHERE time > now() - INTERVAL '1 hour'
 GROUP BY bucket, host
 ORDER BY bucket;

SELECT add_retention_policy('cpu', INTERVAL '7 days');  -- drop old chunks
```

System	What it is	Good for
TimescaleDB	Postgres extension: still SQL, joins, the usual tools	metrics next to business data, teams that know Postgres
InfluxDB	purpose-built TSDB	IoT, sensors, high ingest of events
Prometheus TSDB	local store inside Prometheus, pull-based, PromQL	infrastructure monitoring; keeps **15 days** by default

COMMON MISTAKES

- Putting `user_id`, `request_id` or full URLs in metric labels.
- Keeping raw points forever instead of downsampling.
- Deleting old data row by row in plain Postgres instead of dropping partitions or chunks.
- Downsampling to the average only, which hides spikes; keep min and max (or percentiles) too.

6 Key terms

Time series: values over time for one set of labels

Downsampling: roll raw points up into coarser summaries

Retention: how long data is kept before it is dropped

Hypertable: TimescaleDB table chunked by time

Continuous aggregate: a rollup kept up to date automatically

Cardinality: number of unique series

Chunk: one time range of a table

Recording rule: Prometheus precomputed series

time_bucket: groups timestamps into fixed buckets

7 Interview questions

Q1. Why use a time-series database instead of Postgres for metrics? BASIC

It is built for append-heavy timestamped data: time-chunked storage, column compression that saves over 90%, cheap retention by dropping chunks, built-in downsampling, and fast time-range aggregates. Postgres works at small scale, and TimescaleDB adds these features to it.

Q2. What is downsampling? BASIC

Rolling raw points up into coarser summaries, for example 1-minute points into hourly min, avg and max. Raw is kept for a short window (a week), summaries for a long one (a year), cutting history about 60×.

Q3. Why is deleting old data cheap in a TSDB? BASIC

Data is chunked by time, so expiring a month means dropping a few whole chunks, a file operation. In plain Postgres it is a DELETE over billions of rows, followed by bloat and vacuuming.

Q4. 10,000 servers report 20 metrics every 10 seconds. How many points per day? SCENARIO

10,000 × 20 × 8,640 = about 1.7 billion points a day (about 20,000 per second). At about 1.5 bytes per sample, that is roughly 2.6 GB a day compressed: fine for a TSDB, painful as plain rows.

Q5. Prometheus memory doubled after a deploy. What do you check first? SCENARIO

The active series count, and which metric grew. A new label with unbounded values (user ID, request ID, raw path) is the usual cause. Drop or bucket that label, and put the IDs in logs or traces.

Q6. Dashboards over the last 90 days are slow. Fix? SCENARIO

Query a continuous aggregate (hourly rollup) instead of raw points: 60× fewer rows. Keep raw for a week only, and compress older chunks.

Q7. Is it fine to downsample to the hourly average only? TRICKY

No. An average hides spikes: a 5-minute CPU spike to 100% inside an otherwise idle hour shows as about 10%. Keep min and max, or percentiles, alongside the average.

Q8. Why not just add more labels to make metrics more useful? TRICKY

Labels multiply series. 10,000 hosts × 50 endpoints is already 500,000 series; adding a 1-million-value label can mean millions more. Each series costs memory and index space, so bounded labels only.

Next: SD-E-019 · Leader-Follower Replication backendroadmap.com